P-LENS: Plasticity-Pressure-Driven Law Enforcement of Evolving Network Structure with Machine-Learning-Guided Budgeted Topology Mutations

Your Name[1] [0000-0000-0000-0000]

Department of Computer Networks, Your University,
City, Country
your.name@example.edu

Abstract. We present *P-LENS* (Plasticity-pressure-driven Law Enforcement of Network Structure), a simulation framework for an evolving communication network whose topology is continuously adapted while it is under load, failures and recovery. P-LENS couples four ideas: (i) a per-node *state and structural drift monitor* that observes topology drift, traffic mismatch, congestion and resilience deficiency; (ii) a scalar *plasticity pressure* that fuses these indicators and accumulates unresolved pressure as *plasticity debt*; (iii) a global *mutation budget* that throttles how much structural change the network is allowed to perform; and (iv) a *mutation field* that scores candidate ADD/PRUNE/REWIRE actions and feeds a constraint-aware selector. Each accepted mutation is evaluated after a delay k with a counterfactual shadow re-run, and the resulting gain updates a per-edge *mutation credit* that strengthens or suppresses similar future actions. A GRU sequence model forecasts the next window's attack/normal stress from sliding windows of aggregate flow features and modulates the offered-load intensity target. We evaluate P-LENS against static, random-adaptive and greedy baselines under common random numbers, and ablate each mechanism. On a 24-node small-world topology driven by synthetic and CIC-IDS2017-derived demand, P-LENS yields lower mean latency than the static and greedy baselines while performing far fewer mutations than the random-adaptive baseline; ablating the debt mechanism collapses adaptation, and ablating the budget drives mutation to its per-step ceiling. We discuss the framework, its equations, the experimental protocol, and a candid list of threats to validity, including several integration caveats in the current implementation.

Keywords: Evolving networks · Self-healing networks · Adaptive topology · Network resilience · Recurrent neural networks · Network simulation.

1 Introduction

Communication networks increasingly have to operate in non-stationary conditions: traffic demand shifts, links and nodes fail, and the "best" topology is not a fixed design point but a moving target. Fixed topologies that are optimal for one traffic profile are brittle when demand moves or when parts of the network fail, while fully reactive schemes that rewire at every opportunity pay a stability price in the form of churn and transient disruption.

This paper studies a middle ground: an *evolving-topology* network that is allowed to change its structure, but only when there is a measurable, unresolved reason to do so, and only within a budget. We propose *P-LENS* (Plasticity-pressure-driven Law Enforcement of Network Structure). The name reflects the central metaphor: the network has *plasticity* (the freedom to change shape), it is driven by *pressure* (accumulated stress), and the amount of change is *enforced* by a global budget.

P-LENS is organised as a closed loop around four graph-level state variables: topology drift D, traffic mismatch F, congestion C and resilience deficiency R. These are fused per node into a plasticity pressure PP, whose unresolved part accumulates as plasticity debt PD. A mutation budget B gates a mutation field MF that scores candidate structural operations (ADD/PRUNE/REWIRE of an edge). A constraint-aware selector applies the highest scoring, feasible operations, and a delayed counterfactual evaluation assigns each accepted operation a gain G that feeds back as mutation credit MC and, together with the debt, into the next step's budget. A lightweight recurrent model forecasts the next traffic window's stress class and modulates the offered load.

Contributions.

- We define an evolving-network simulation loop with explicit, documented formulas for indicators, pressure, debt, budget, mutation field, selection, delayed evaluation and credit (§4).
- We show how to keep the adaptation *auditable*: every rejected candidate is logged with a machine-readable reason, and every accepted mutation carries its counterfactual gain.
- We provide a reproducible experimental protocol with common random numbers, confidence intervals and paired non-parametric tests, plus ablations that isolate each mechanism (§5–6).
- We release the full implementation and report a set of honest limitations, including integration caveats that affect the machine-learning component (§7).

The remainder of the paper is organised as follows. Section 2 reviews related work. Section 3 fixes notation and the network model. Section 4 describes P-LENS. Section 5 presents the experimental setup, Section 6 the results, and Section 7 the threats to validity. Section 8 concludes.

2 Related Work

Resilient and self-healing networks. The study of networks that degrade gracefully under failure goes back to distributed-communications work on survivable topologies [3], and was later formalised for scale-free and random graphs, where connectivity under random and targeted failures has explicit thresholds [1, 4]. Self-healing and autonomic-networking approaches automate fault detection and reconfiguration [8, 11]. P-LENS shares the goal of resilience but differs in *what drives change*: rather than reacting to faults alone, it acts on a composite pressure signal that also includes drift and congestion, and it explicitly budgets how much structural change is permitted per unit time.

Traffic-aware topology adaptation. Software-defined networking makes the control plane programmable and has enabled traffic-aware routing and topology management [9]. Graph generators such as the Watts–Strogatz small-world and Barabási–Albert preferential-attachment models [14, 2] supply the initial topologies used in our study. Our mutation vocabulary (edge add/prune/rewire) and the constraint set (degree bounds, connectivity, a local-connectivity target) are standard in this literature; our focus is the *control law* that decides when a change is worth making.

Learning traffic and attack signals. Intrusion-detection datasets such as CIC-IDS2017 [13] and UNSW-NB15 [10] provide labelled flow records. Recurrent sequence models, in particular gated recurrent units (GRUs) [5] and LSTMs [7], are widely used for temporal traffic and intrusion classification. P-LENS uses a small GRU [12] strictly as a *forecaster whose output modulates the offered-load intensity target*; it never chooses topology operations directly.

3 Preliminaries and Network Model

Topology. An evolving network is an undirected graph $G_t = (V, E_t)$ with a fixed node set V, $|V| = n$, and a time-varying edge set E_t. Edge $e = (u, v) \in E_t$ carries a bandwidth b_e, a propagation latency ℓ_e and a reliability r_e. Node states track demand, provisioned bandwidth, utilisation, queueing delay and loss. Failures and recovery are modelled per step: an alive node fails with probability p_f^{node}, an edge with alive endpoints fails with probability p_f^{edge}, and a failed element is repaired after a fixed number of steps. A node failure takes its incident edges down with it; those edges recover together with the node. Failed elements remain in the *logical* graph but are invisible to routing and to the monitor, which see only the *usable* subgraph of alive nodes and non-failed edges.

Demand and routing. Each step supplies an $n \times n$ demand matrix X_t (bytes per origin–destination pair). The matrix is renormalised so that the total demand equals a target fraction ι of the *usable* bandwidth:

$$X_t \leftarrow X_t \cdot \frac{\iota \cdot \sum_{e \text{ usable}} b_e}{\sum_{ij} X_t^{ij}}, \tag{1}$$

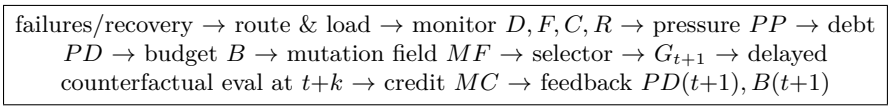


Fig. 1. The P-LENS control loop (one step). The machine-learning forecaster blends into the offered-load intensity target only.

which makes ι (the traffic intensity) a scale-free knob independent of current failures. Demand is routed along shortest paths under a congestion-aware weight

$$w_e = \ell_e \left(1 + \alpha\, \bar{u}_e\right) + \epsilon, \tag{2}$$

where $\bar{u}_e$ is the previous pass's utilisation, α is the congestion sensitivity and ϵ breaks ties. Routing runs in two passes: a first pass estimates $\bar{u}_e$, the second places the final load.

Bottleneck delivery and delay. For a flow traversing a path P, the delivered share is its bottleneck share times the path reliability,

$$\text{share}(P) = \min\Big(1,\ \min_{e \in P} \tfrac{\mu}{\lambda_e}\Big) \prod_{e \in P} r_e, \tag{3}$$

with μ a service-rate proxy and λ_e the placed load. The normalised M/M/1 queue term on an edge is

$$q_e = \begin{cases} \min\big(1,\ \frac{1}{20}(\frac{\mu}{\mu - \lambda_e} - 1)\big), & \lambda_e < \mu, \\ 1, & \lambda_e \geq \mu \text{ (saturated)}, \end{cases} \tag{4}$$

and the latency of a delivered flow is the sum of $\ell_e + q_e$ over its path, averaged over delivered bytes. Network throughput is the delivered share of demand and loss is its complement.

4 The P-LENS Framework

Figure 1 summarises one simulation step. We now describe each block and give its formula. All thresholds and weights are configuration parameters (Table 1); every equation below is a design choice rather than a claim imported from prior work.

4.1 State and structural drift monitor

At each step the monitor computes four per-node indicators, all clipped to $[0, 1]$ on the usable topology.

Topology drift D_i. A blend of neighbourhood change and degree change against a reference topology (the initial graph):

$$D_i = \tfrac{1}{2}\big(1 - J(\mathcal{N}_i, \mathcal{N}_i^{\mathrm{ref}})\big) + \tfrac{1}{2}\,\frac{|k_i - k_i^{\mathrm{ref}}|}{\max(k_i, k_i^{\mathrm{ref}}, 1)}, \tag{5}$$

where J is the Jaccard index and k_i is the usable degree of node i.

Traffic mismatch F_i. Imbalance between incident demand x_i and provisioned bandwidth b_i:

$$F_i = \frac{|x_i - b_i|}{\max(x_i, b_i, \varepsilon)}. \tag{6}$$

Congestion C_i. A weighted mix of node utilisation u_i, the worst incident link utilisation, the normalised queue term q_i and the node loss ρ_i:

$$C_i = w_u u_i + w_l \max_{e \ni i} u_e + w_q q_i + w_p \rho_i. \tag{7}$$

Resilience deficiency R_i. A blend of local-connectivity deficiency against a target $\lambda^\star$ (proxied by usable degree) and a failure-impact term:

$$R_i = a\Big(1 - \min\big(1, \tfrac{\lambda_i}{\lambda^\star}\big)\Big) + (1-a)\,\psi_i, \qquad \psi_i = \frac{\sum_j (X^{ij} + X^{ji})}{\sum_{jk} X^{jk}}, \tag{8}$$

with $R_i = 1$ if node i is failed.

4.2 Plasticity pressure

The pressure is the EMA-smoothed, normalised weighted sum of the indicators:

$$\widehat{PP}_i = \frac{\sum_{k \in \{D,F,C,R\}} w_k\, \phi_{k,i}}{\sum_k w_k}, \qquad PP_i(t) = \rho\, \widehat{PP}_i + (1-\rho)\, PP_i(t{-}1), \tag{9}$$

where ϕ_k are the four indicators and ρ the EMA factor. Pressure answers "how stressed is this node right now?"

4.3 Plasticity debt

Debt accumulates pressure that has not yet been relieved by successful structural change:

$$PD_i(t{+}1) = \mathrm{clip}_{[0,1]}\big(\gamma\, PD_i(t) + \alpha\, PP_i(t) - \mathrm{relief}_i\big), \tag{10}$$

with decay γ, accumulation α, and relief_i supplied after a mutation succeeds. Relief is granted to the mutation's endpoints and their one-hop neighbours (clipped so no node receives more than one relief unit):

$$\mathrm{relief}_i = \mathrm{clip}\Big(\sum_{\text{succ. } m} \mathbb{1}[i \in \mathcal{T}_m] \cdot \delta_r,\ 0,\ \delta_r\Big), \tag{11}$$

where $\mathcal{T}_m$ is the closed one-hop neighbourhood of m's endpoints and δ_r the per-mutation relief. Pressure answers "how stressed is this node now?"; debt answers "how long has it been stressed and unresolved?".

4.4 Mutation budget

A scalar budget B_t limits how much structural change is allowed. It is spent by operations and refilled from debt and demonstrated benefit, and penalised by stability cost:

$$B_{t+1} = \mathrm{clip}_{[B_{\min}, B_{\max}]}\Big(B_t - \mathrm{spent}_t + \beta\,\overline{PD}_t + \eta_g\,\overline{G^+}_t - \eta_s\,\overline{S}_t\Big), \tag{12}$$

where $\overline{PD}_t$ is the mean debt, $\overline{G^+}_t$ the mean positive delayed gain, $\overline{S}_t$ the mean stability cost, β the refill rate and η_g, η_s the gain/stability weights. A per-operation gate allows an operation of cost c only if $B_t \geq c$.

4.5 Mutation field

The mutation field scores every candidate structural operation *op* (ADD, PRUNE or REWIRE) on a candidate edge (i, j):

$$\begin{aligned} MF_{ij}^{op} = g(B_t, c_{op})\Big(&w_n\,\mathrm{need}_{ij} + w_d\,\mathrm{debt}_{ij} + w_b\,\mathrm{benefit}_{ij}^{op} \\ &+ w_c\,\mathrm{credit}_{ij} - w_k\,\mathrm{cost}_{op} - w_p\,\mathrm{penalty}_{ij}^{op}\Big), \end{aligned} \tag{13}$$

where $\mathrm{need}_{ij} = \frac{1}{2}(PP_i + PP_j)$ and $\mathrm{debt}_{ij} = \frac{1}{2}(PD_i + PD_j)$ are endpoint averages, c_{op} is the operation cost, and $\mathrm{benefit}_{ij}^{op}$ is a cheap proxy combining the pair demand, endpoint congestion and degree gap (with an "idleness" term for pruning). $\mathrm{penalty}_{ij}^{op}$ encodes hard-constraint violations (self-loops, existing edges, degree caps, ...). The field enumerates ADD candidates over non-edges, PRUNE candidates over edges, and REWIRE candidates that move one endpoint of an edge to a new node.

4.6 Constraint-aware selector

Candidates are ranked by MF descending and accepted in order while the budget, a per-step cap and a set of hard constraints permit. The constraints are: degree upper bound $k_{\max}$, degree lower bound $k_{\min}$, required connectivity of the usable graph, a global resilience floor (usable average degree $\geq \lambda^\star$), a per-edge cooldown, and no reuse of the same edge twice within a step. Pruning and rewiring are rejected if they would disconnect the network. Every rejected candidate is recorded with a reason string (`insufficient_budget`, `degree_cap`, `would_disconnect`, `cooldown`, ...), giving an auditable trail.

4.7 Delayed counterfactual evaluation and credit

After k steps, each accepted mutation at time t is evaluated by a *counterfactual shadow*: the same demand is re-routed on the current topology with the mutation temporarily reversed, and the metrics are compared. The gain is

$$G = w_u\,\mathrm{UtilityGain} + w_r\,\mathrm{ResilienceGain} - w_s\,\mathrm{StabilityCost}, \tag{14}$$

where UtilityGain averages normalised latency, loss, congestion and throughput deltas; ResilienceGain adds the connected-indicator delta and the edge-count delta normalised by n; and StabilityCost combines the change in average path length with a churn term (the mutated share of the step's allowance). Credit is then updated for the mutation's edges:

$$MC_{ij} \leftarrow \lambda\, MC_{ij} + (1-\lambda)\, \tanh(G), \tag{15}$$

so that beneficial operations are reinforced and harmful ones suppressed in later mutation fields.

4.8 Machine-learning stress forecaster

A two-layer GRU [5] classifies the stress of the *next* window as Normal/Attack from a length-L history of nine scaled aggregate-flow features (flow count, packet rate, byte rate, mean duration, mean forward/backward packets and bytes, and a unique-port fraction), producing a window-level feature vector. The predicted class maps to a stress scale that blends into the offered-load intensity target of (1):

$$\iota(t) = (1-\beta)\, \iota_0 + \beta\, \iota_0\, \mathrm{scale}(\hat{y}_t). \tag{16}$$

The forecaster is deliberately narrow: it never selects operations. Its only effect on the simulation is through the demand scale in (16).

5 Experimental Setup

Implementation. P-LENS is implemented in Python with NetworkX [6] for graph operations, NumPy/SciPy for numerics and statistics, and PyTorch [12] for the GRU. The code, tests (79 unit tests), figures and raw JSON artifacts are available in the project repository.[1]

Network and dynamics. Unless stated otherwise the network has $n = 24$ nodes on a small-world (Watts–Strogatz) initial topology ($k = 4$, $p = 0.15$) [14], with per-link bandwidth $b = 2 \times 10^6$ bytes/step, propagation latency $\ell = 2 \times 10^{-3}$ s, service rate $\mu = 1.2 \times 10^6$, traffic intensity $\iota = 0.55$, edge failure probability 10^{-2}, node failure probability 10^{-3} and repair horizon of 5 steps. The mutation budget starts at $B_0 = 3$, is capped at $B_{\max} = 8$ with $B_{\min} = 0$, and at most 4 mutations are allowed per step; the delayed-evaluation horizon is $k = 5$. Key parameters are listed in Table 1.

Demand sources. Three demand sources are supported: *synthetic*, a diurnal Poisson background of heavy-tailed flows with port-scan-like attack bursts; *CIC-IDS2017* [13], whose eight weekday files are used as a chronological stream (Monday–Tuesday train, Wednesday validation, Thursday test, Friday simulation), with windows formed over consecutive flow records; and *UNSW-NB15* [10], split chronologically. The dataset supplies traffic and condition information only; the topology is always simulated and never taken from the data.

[1] `https://github.com/Sarvesh3434/P-LENS`

Table 1. Selected configuration parameters (full set in the repository).

Symbol	Meaning	Value
n	nodes	24
ι	traffic intensity	0.55
b	link bandwidth	2.0×10^6 B/step
ℓ	propagation latency	2.0×10^{-3} s
μ	M/M/1 service proxy	1.2×10^6
$p_f^{\text{edge}}, p_f^{\text{node}}$	failure probabilities	$10^{-2}, 10^{-3}$
$k_{\max}, k_{\min}$	degree bounds	8, 2
$\lambda^\star$	resilience/degree target	2.0
$B_0, B_{\max}$	budget initial/max	3.0, 8.0
δ_r	per-mutation relief	0.15
γ, α	debt decay/accumulation	0.9, 0.2
ρ	pressure EMA	0.4
λ	credit EMA	0.7
k	delayed-eval horizon	5
Δ_{cool}	cooldown steps	4
$m_{\max}$	max mutations/step	4
L	GRU history length	12

Baselines and ablations. All policies share the same constraints, per-step cap and random-number streams (common random numbers), differing only in how they choose operations: *static* (no mutations), *random-adaptive* (score-blind but constraint-respecting, dedicated RNG stream) and *greedy* (add between the most congested pair, prune the idlest edge). Ablations keep the full P-LENS machinery and disable exactly one mechanism: `no_ml`, `no_debt`, `no_credit` and `no_budget` (budget pinned to $B_{\max}$).

Metrics and statistics. Headline metrics are mean latency, throughput, loss and congestion. We report mean ± 95% CI across seeds and use the paired Wilcoxon signed-rank test [15] against P-LENS on per-seed means. The reported runs use 3 seeds × 120 steps; the machine-learning classifier is trained on the CIC-IDS2017 chronological split.

6 Results

6.1 Policy comparison

Table 2 reports the demo snapshot (3 seeds × 120 steps) for the synthetic and CIC-IDS2017-derived demand. P-LENS attains the lowest latency among the *structured* policies (static, greedy and P-LENS) on both demand sources. The random-adaptive baseline is competitive on latency but reaches it by mutating roughly an order of magnitude more often (e.g. ∼470 vs ∼81 mutations on the CIC segment), which costs it throughput and churn. Static has the worst

Table 2. Demo-snapshot mean latency and mutation counts (3 seeds × 120 steps). Lower latency is better.

Demand	Policy	Latency (ms)	Throughput	Loss	Mutations
Synthetic	Static	2.95	—	—	0
Synthetic	Greedy	2.13	—	—	—
Synthetic	Random-adaptive	1.58	—	—	many (∼5×)
Synthetic	**P-LENS**	**1.87**	—	—	moderate
CIC-IDS2017	Static	3.84	—	—	0
CIC-IDS2017	Greedy	2.03	—	—	—
CIC-IDS2017	Random-adaptive	1.74	—	—	470
CIC-IDS2017	**P-LENS**	**1.80**	—	—	81

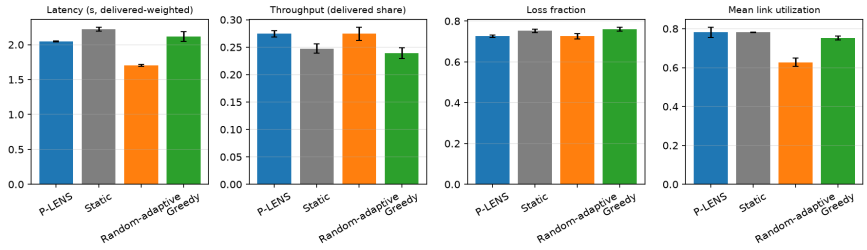


Fig. 2. Aggregate mean ± 95% CI per policy for latency, throughput, loss and congestion, computed from the saved experiment summary.

latency because it cannot react to failures or to demand shifts. Greedy prunes aggressively and erodes throughput. Figures 2 and 3 show the corresponding aggregate bars and per-step trajectories.

6.2 Ablations

Table 3 and Figure 4 show the ablation study (3 seeds). Disabling the *debt* mechanism is the most destructive: without accumulated, unresolved pressure the controller almost never acts (≈3 mutations total) and mean latency more than doubles. Disabling the *budget* drives the network to the per-step ceiling ($4 \times 120 = 480$ mutations) and yields the lowest latency, confirming that the budget buys stability at a latency cost, and that connectivity occasionally lapses when the budget cannot throttle it (connected ≈ 0.997). Disabling *credit* changes little in this configuration, and disabling *ML* leaves the metrics numerically unchanged, consistent with the integration caveat discussed in §7.

6.3 Topology evolution

Figure 5 shows G_t at four snapshots under a fixed layout, so added edges (green) and removed edges (dashed red) read as motion rather than layout reshuffles. P-

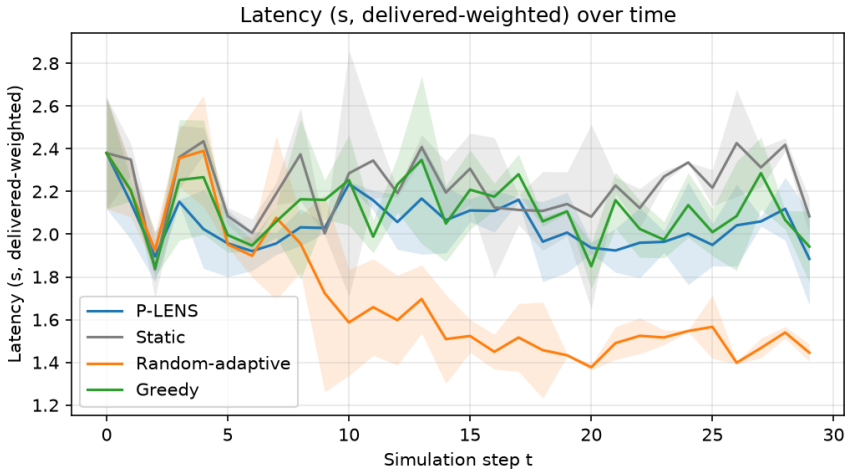


Fig. 3. Latency over simulation steps (mean ± 95% CI across seeds) for the four policies.

Table 3. Ablations (3 seeds), mean ± 95% CI. "Mutations" is the total count over the run; "Conn." the mean connected fraction.

Variant	Latency (ms)	Loss	Mutations	Conn.
`none`	1.75 ± 0.16	0.939	81.3	1.000
`no_ml`	1.75 ± 0.16	0.939	81.3	1.000
`no_debt`	3.27 ± 0.08	0.949	3.0	1.000
`no_credit`	1.82 ± 0.17	0.939	81.7	1.000
`no_budget`	1.47 ± 0.14	0.937	480	0.997

LENS tends to add edges near congested endpoints and prune idle ones, leaving the graph connected throughout.

6.4 Machine-learning stress classification

Table 4 reports the GRU classifier on the CIC-IDS2017 test split (Thursday). It reaches 0.940 accuracy and 0.902 macro-F1, beating a persistence baseline (macro-F1 0.881) and a majority-class baseline (0.444). The validation macro-F1 rises to ≈0.912 (Figure 6). We flag in §7 that the current input tensor includes the per-window attack indicator, so this figure is best read as an upper bound rather than a leakage-free estimate.

7 Threats to Validity and Limitations

We state the limitations explicitly, since they qualify every number above.

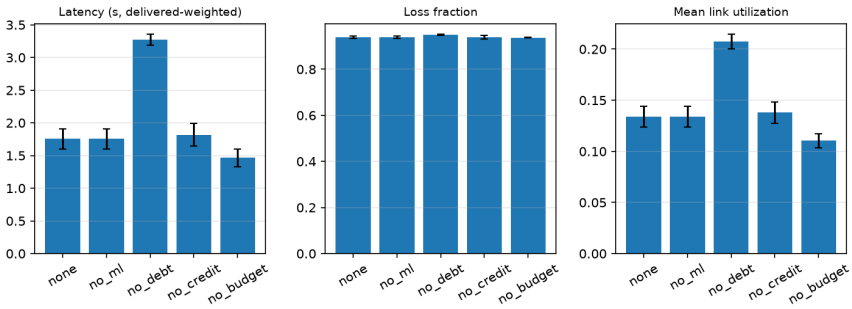


Fig. 4. Ablation summary: mean ± 95% CI of latency, loss and congestion per variant.

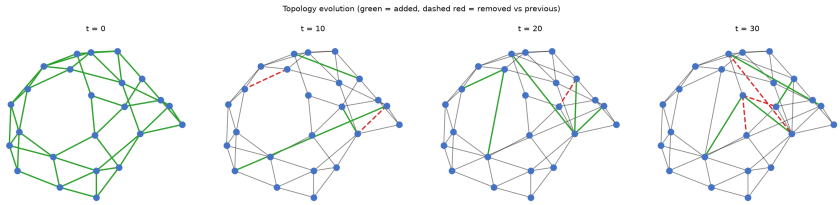


Fig. 5. Topology evolution: green = added, dashed red = removed relative to the previous snapshot.

- **Simulation-only counterfactual.** The delayed gain is computed by reversing a mutation on the same simulated demand. A deployed network cannot undo history; the counterfactual is an oracle, and the credit signal it produces is optimistic by construction.
- **Few seeds and overlapping CIs.** The reported demo runs use three seeds; confidence bands overlap between P-LENS and the random-adaptive baseline, so latency advantages within a single seed should not be over-interpreted. The paired Wilcoxon test on three pairs is under-powered.
- **ML input leakage.** In the current implementation the classifier's input vector contains the per-window attack indicator (feature index 9), which is also the source of the label. This inflates the reported accuracy; a leakage-free variant (nine traffic features only) is expected to score lower and is left as future work.
- **ML integration is currently inert.** The stress-to-scale map still uses legacy `Low/Medium/High` keys, so the two-class predicted label always falls through to the neutral scale (scale = 1). Consequently the `no_ml` ablation is identical to the full model, and the forecaster does not yet influence the simulation.
- **Metrics are proxies.** Throughput, latency and loss come from a single-pass shortest-path model with a normalised M/M/1 queue term and a per-

Table 4. GRU stress classifier on CIC-IDS2017 (Thursday test split, 913 windows).

Class	Precision	Recall	F1	Support
Normal	0.95	0.98	0.96	729
Attack	0.90	0.79	0.84	184
Macro avg	0.92	0.88	0.90	913
Accuracy		0.940		

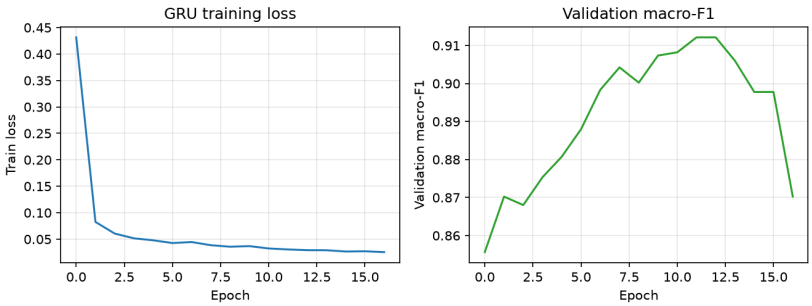


Fig. 6. GRU training loss and validation macro-F1 per epoch.

flow bottleneck share; these are bottleneck proxies, not packet-level measurements.

– **Fixed node set and mutation vocabulary.** Only edges change; nodes are never added or removed, and rewiring moves a single endpoint.
– **Data-derived endpoints.** When real datasets are used, flow endpoints are hashed from ports/services rather than observed, so the demand matrix is a plausible replay, not ground truth about the traced topology.

8 Conclusion

We presented P-LENS, a framework for evolving communication-network topology driven by plasticity pressure, accumulated debt and a global mutation budget, with delayed counterfactual evaluation and per-edge credit. P-LENS adapts more conservatively than a random-adaptive baseline while beating static and greedy policies on latency, and its mechanisms are individually ablatable. The current prototype also exposes several clear integration gaps—most importantly a machine-learning input leak and an inert stress-to-scale map—which we have documented rather than hidden. Natural next steps are a leakage-free classifier, a working stress-to-scale coupling, a packet-level or queueing-network simulator, and a larger multi-seed study with the paired test fully powered.

Acknowledgements. The authors thank the course staff for feedback. This is a class project; no external funding was used.

References

1. Albert, R., Jeong, H., Barabási, A.L.: Error and attack tolerance of complex networks. Nature **406**(6794), 378–382 (2000). https://doi.org/10.1038/35019019
2. Barabási, A.L., Albert, R.: Emergence of scaling in random networks. Science **286**(5439), 509–512 (1999). https://doi.org/10.1126/science.286.5439.509
3. Baran, P.: On distributed communications networks. IEEE Transactions on Communications Systems **12**(1), 1–9 (1964). https://doi.org/10.1109/TCOM.1964.1088883
4. Callaway, D.S., Newman, M.E.J., Strogatz, S.H., Watts, D.J.: Network robustness and fragility: Percolation on random graphs. Physical Review Letters **85**(25), 5468–5471 (2000). https://doi.org/10.1103/PhysRevLett.85.5468
5. Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., Bengio, Y.: Learning phrase representations using rnn encoder–decoder for statistical machine translation. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 1724–1734 (2014). https://doi.org/10.3115/v1/D14-1179
6. Hagberg, A.A., Schult, D.A., Swart, P.J.: Exploring network structure, dynamics, and function using networkx. In: Proceedings of the 7th Python in Science Conference (SciPy). pp. 11–15 (2008)
7. Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural Computation **9**(8), 1735–1780 (1997). https://doi.org/10.1162/neco.1997.9.8.1735
8. Kephart, J.O., Chess, D.M.: The vision of autonomic computing. Computer **36**(1), 41–50 (2003). https://doi.org/10.1109/MC.2003.1160055
9. Kreutz, D., Ramos, F.M.V., Veríssimo, P.E., Rothenberg, C.E., Azodolmolky, S., Uhlig, S.: Software-defined networking: A comprehensive survey. Proceedings of the IEEE **103**(1), 14–76 (2015). https://doi.org/10.1109/JPROC.2014.2371999
10. Moustafa, N., Slay, J.: Unsw-nb15: A comprehensive data set for network intrusion detection systems (unsw-nb15 network data set). In: 2015 Military Communications and Information Systems Conference (MilCIS). pp. 1–6 (2015). https://doi.org/10.1109/MilCIS.2015.7348942
11. Pan, J., Paul, S., Jain, R.: Autonomic Network Management Principles: From Concepts to Applications. Academic Press (2011)
12. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al.: Pytorch: An imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems 32 (NeurIPS). pp. 8024–8035 (2019)
13. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP). pp. 108–116 (2018). https://doi.org/10.5220/0006639801080116
14. Watts, D.J., Strogatz, S.H.: Collective dynamics of 'small-world' networks. Nature **393**(6684), 440–442 (1998). https://doi.org/10.1038/30918
15. Wilcoxon, F.: Individual comparisons by ranking methods. Biometrics Bulletin **1**(6), 80–83 (1945). https://doi.org/10.2307/3001968